PENS-KAIT 2026

Context-Aware Robot Control Dashboard for Edge Deployment on Raspberry Pi 5

Weekly Seminar Progress Report

March 16, 2026

Author:	Muhammad Tsaqif Mukhayyar
Supervisors:	Prof. Kosuke Takano (Kanagawa Institute of Technology) Dr. Eng. Idris Winarno (PENS)
Affiliation:	PENS-KAIT Joint Research (Politeknik Elektronika Negeri Surabaya & Kanagawa Institute of Technology)

Abstract

This report presents the current state of PENS-KAIT 2026: a browser-based, real-time robot control dashboard running on Raspberry Pi 5 with *context-aware object detection*. The system implements a **two-phase intelligence pipeline**: Phase 1 uses GoogLeNet (Places365, 365 scene categories) to identify the robot's scene, and Phase 2 dynamically reconfigures a YOLOWorld detection model based on a scene-to-class database. A **dual-vocabulary context architecture** (COCO-80 and Objects365) was completed, providing on average 1.9× more detectable object categories per scene. Benchmark results show TFLite float16 achieves the fastest inference at **302 ms** on Raspberry Pi 5 CPU, compared to 420 ms (ONNX) and 460 ms (PyTorch). Hardware storage limitations currently restrict deployment of the full Objects365 model set; procurement of a high-capacity NVMe SSD is planned. Extended results and experiments are hosted on **Google Drive** and **Google Colab**.

Source code: `/home/takanolab/yahboom_control/PENS-KAIT 2026/`
Mermaid diagram sources: `docs/seminar/diagrams/`

Contents

1 Introduction & Motivation **2**

2 System Architecture Overview **2**
2.1 High-Level Block Diagram . . . 3
2.2 Threading Model . . . 4

3 Hardware Platform **4**
3.1 Yahboom Raspbot + Raspberry Pi 5 . . . 4

4 Two-Phase Intelligence Pipeline **5**
4.1 Pipeline Overview . . . 6
4.2 Phase 1: Scene Recognition . . . 7
4.3 Phase 2: Context-Aware YOLO Detection . . . 7

5 Dual-Vocabulary Context Architecture **8**
5.1 Motivation: The Vocabulary Ceiling Problem . . . 8
5.2 Database Schema . . . 8
5.3 Vocabulary Coverage Analysis . . . 9

6 YOLO Model Benchmark **9**
6.1 Model Formats Evaluated . . . 9

7 Communication Architecture **10**
7.1 Socket.IO Event Map . . . 11
7.2 Camera Streaming: MJPEG vs. WebRTC . . . 11

8 Frontend Architecture **11**
8.1 Component Overview . . . 12

9 Alert Rules Engine **12**

10 Docker Deployment Architecture **13**
10.1 Build & Run Commands . . . 14

11 Storage Limitation & Procurement Plan **14**
11.1 Interim Solution: Google Drive & Google Colab . . . 15
11.2 Procurement Plan . . . 15

12 Performance Summary **15**

13 Current Limitations & Future Work **16**

14 Conclusion **16**

A File Structure Reference **16**

B Environment Variables **17**

1 Introduction & Motivation

Remote teleoperation of mobile robots increasingly demands *intelligent perception*: the operator should not need to manually select which objects the robot is watching for as it moves between environments. Existing platforms treat teleoperation and computer vision as separate concerns, requiring manual reconfiguration when the robot transitions from, say, a kitchen to a hospital corridor.

PENS-KAIT 2026 addresses this gap through a **context-aware detection pipeline** that:

1. Continuously recognises the robot's scene using a lightweight scene classifier (Phase 1).
2. Automatically reconfigures the object detector with scene-appropriate classes (Phase 2), drawing from either COCO-80 or the richer Objects365 vocabulary (365 fine-grained categories).
3. Provides real-time motor, servo, LED, and buzzer control through a React web dashboard with Socket.IO.

Research Affiliation

This is a joint research project between **Politeknik Elektronika Negeri Surabaya (PENS)**, Indonesia and **Kanagawa Institute of Technology (KAIT)**, Japan.

2 System Architecture Overview

The system follows a three-tier architecture: **hardware**, **backend server**, and **browser client**. All real-time communication is handled by Socket.IO over WebSocket, with MJPEG (or experimental WebRTC) for the camera stream.

2.1 High-Level Block Diagram

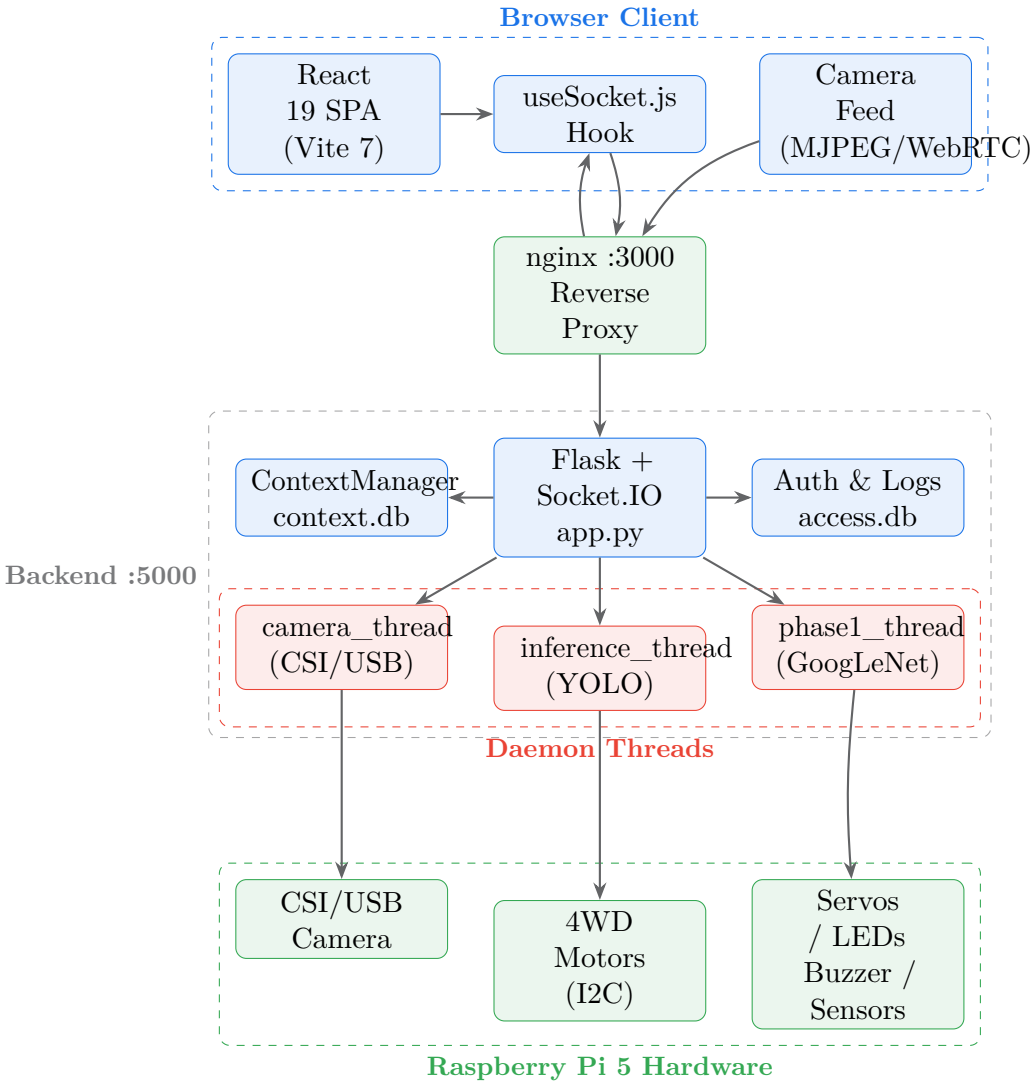

Figure 1: High-level system architecture. All real-time communication flows through Socket.IO; the camera is streamed separately via MJPEG or WebRTC.

Mermaid Source (system_architecture.mmd)

```
graph TB
  subgraph Browser["Browser Client"]
    UI["React 19 SPA (Vite 7)"]
    WS_CLIENT["Socket.IO Client"]
    MJPEG["MJPEG / WebRTC Camera Feed"]
  end
  subgraph Backend["Backend Flask :5000"]
    APP["app.py -- Flask + Socket.IO"]
    CM["ContextManager (context.db)"]
    Threads["camera | inference | phase1 | sensor | broadcast"]
  end
  subgraph Hardware["Raspberry Pi 5 Hardware"]
    CAM["CSI/USB Camera"]
    MOTOR["4WD Motors (I2C)"]
```

```
    ...
  end
  Browser <-->|"Socket.IO / WebSocket"| Backend
  Backend <-->|"Raspbot_Lib I2C / rpicam-vid"| Hardware
```

2.2 Threading Model

The backend runs five concurrent daemon threads sharing state via Python `threading.Lock` objects:

Table 1: Backend daemon threads and their responsibilities.

Thread	Rate	Responsibility
`camera_thread`	30 fps target	Reads YUV420 frames from `rpicam-vid` pipe (CSI) or OpenCV (USB); writes to shared `FrameManager`
`inference_thread`	frame-driven	Runs YOLO Phase 2 on new frames; deduplicates; emits `detection_results`
`phase1_thread`	∼1 Hz	GoogLeNet Places365 scene recognition; triggers Phase 2 switch on stable scene
`sensor_thread`	∼3 Hz	Polls ultrasonic, IR, line-tracking via I2C; emits `sensors`
`broadcast_thread`	∼4 Hz	Pushes robot status & system metrics (CPU, RAM, temp) to all clients

3 Hardware Platform

3.1 Yahboom Raspbot + Raspberry Pi 5

Table 2: Target hardware specifications.

Component	Specification
SBC	Raspberry Pi 5 (4 GB or 8 GB LPDDR4X)
CPU	ARM Cortex-A76 × 4, up to 2.4 GHz (aarch64)
OS	Raspberry Pi OS Bookworm (64-bit)
Camera	CSI module (`rpicam-vid`) or USB webcam (OpenCV)
Drive Control	4× DC motors, I2C motor driver (Raspbot_Lib)
Servo	Pan (ID 1) + Tilt (ID 2), 0–180°, I2C
LED	WS2812 RGB strip (7 preset colours + custom RGB)
Buzzer	On/off + configurable pattern
Sensors	Ultrasonic distance, 4-ch line-tracking, IR remote
Storage	microSD (current)

Storage Limitation — Action Required

The Raspberry Pi 5 currently boots from a **microSD card** with limited available capacity. The full **Objects365 frozen model set** (365 scene models, each ∼25–50 MB per format variant) requires several gigabytes of storage that exceeds the current card's free space. The Objects365 data has been prepared and is available on **Google Drive** and **Google Colab** for cloud-side benchmarking. **A high-capacity NVMe SSD (M.2 HAT+ or USB 3.0 enclosure) is planned for procurement** to enable full on-device deployment of all model variants.

4 Two-Phase Intelligence Pipeline

4.1 Pipeline Overview

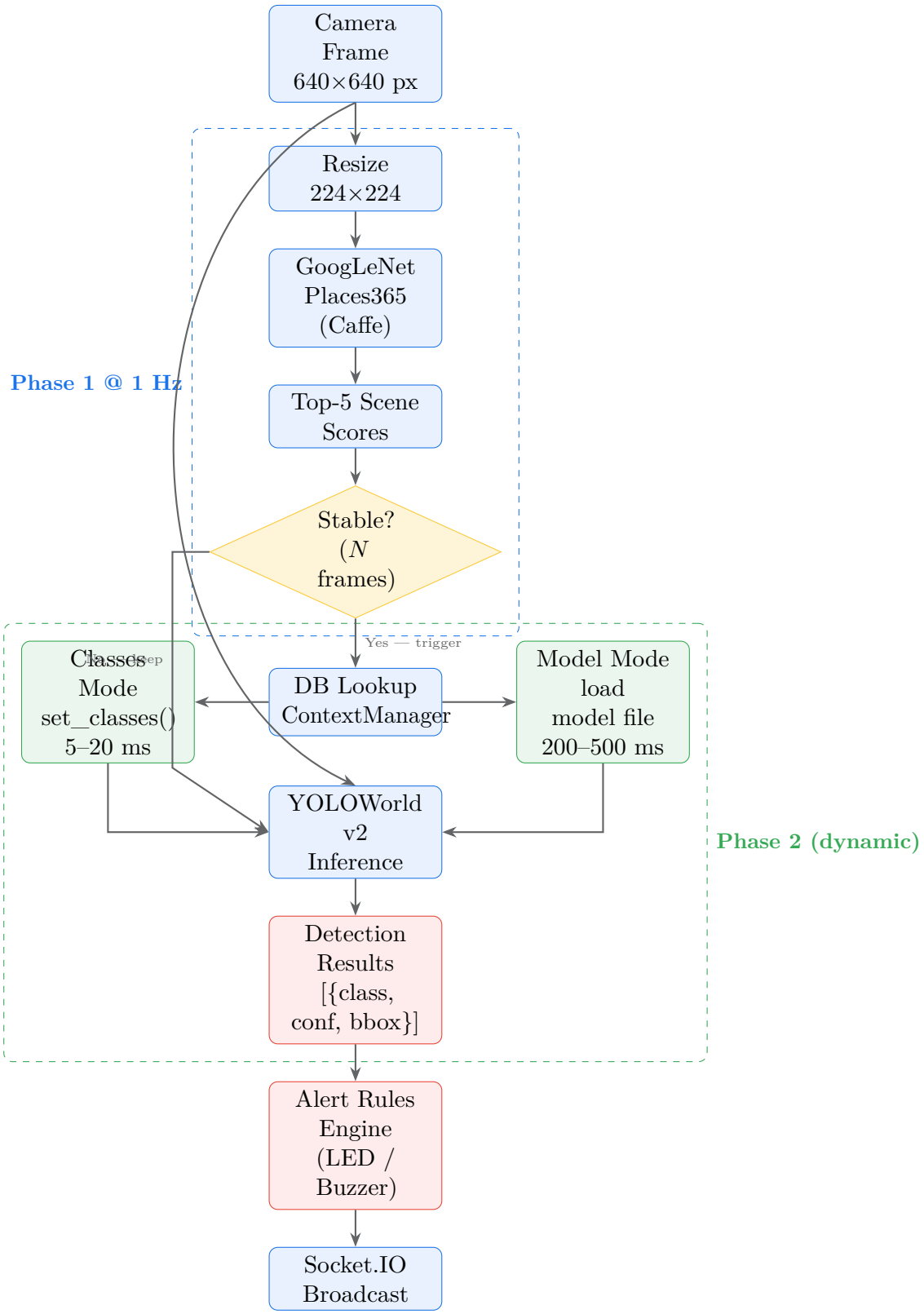

Figure 2: Two-phase context-aware detection pipeline. Phase 1 runs at 1 Hz for scene recognition; Phase 2 adapts YOLO classes based on the stable scene.

Mermaid Source (phase1_phase2_pipeline.mmd)

```
flowchart TD
    CAM["Camera Frame (640x640 px)"]
    subgraph P1["Phase 1 -- Scene Recognition (1 Hz)"]
        P1_DNN["GoogLeNet Places365 (Caffe)"]
        P1_STABLE{"Stable for N frames?"}
    end
    subgraph P2["Phase 2 -- Context-Aware Detection"]
        DB_LOOKUP["ContextManager.get_context_for_scene()"]
        M_CLASS["classes mode: set_classes() ~5-20 ms"]
        M_MODEL["model mode: Load model ~200-500 ms"]
        YOLO["YOLOWorld v2 Inference"]
    end
    CAM --> P1_DNN --> P1_STABLE
    P1_STABLE -->|"Yes"| DB_LOOKUP
    DB_LOOKUP --> M_CLASS & M_MODEL --> YOLO
    CAM --> YOLO --> RESULTS --> ALERT --> EMIT
```

4.2 Phase 1: Scene Recognition

Phase 1 uses **GoogLeNet trained on Places365** (Caffe format, loaded via OpenCV DNN) to classify the camera frame into one of **365 scene categories**. Key parameters:

- Input resolution: 224×224 (resized from the 640×640 capture frame)
- Output: Top-5 scene class scores with probability
- Target run rate: $\sim$1 Hz (triggered every N inference frames)
- Stability threshold: 3 consecutive frames predicting the same scene before triggering Phase 2
- All predictions are logged to `phase1_history` (SQLite) with UUID `infer_id` and `inference_ms`

4.3 Phase 2: Context-Aware YOLO Detection

When Phase 1 declares a stable scene, `_switch_scene()` is invoked:

1. Query `ContextManager.get_context_for_scene(scene, vocabulary=_context_vocab)` — returns the YOLO class list and (optionally) an alternative model file.
2. Switch the active YOLO model to the target classes via one of two modes:

Table 3: Phase 2 switch mode comparison.

Mode	Mechanism	Typical Latency
`classes` (default)	`yolo_model.set_classes(list)`	5–20 ms
`model`	Reload model file from disk	200–500 ms

All switches are logged to `inference_history` with a UUID `switch_id` and timing breakdown (`db_query_ms`, `switch_ms`, `total_ms`).

5 Dual-Vocabulary Context Architecture

5.1 Motivation: The Vocabulary Ceiling Problem

Standard YOLOv8 models operate on the COCO-80 vocabulary, which covers 80 common object categories. Many semantically important objects — `stethoscope`, `whiteboard`, `saxophone`, `cutting board` — have no COCO-80 equivalent, causing Phase 2 to silently ignore them.

The **Objects365** dataset (Shao et al., 2019) provides 365 fine-grained categories that substantially expand coverage for specialised environments.

5.2 Database Schema

Both vocabularies are stored in a single `context.db` file for ACID consistency and minimal Docker volume management:

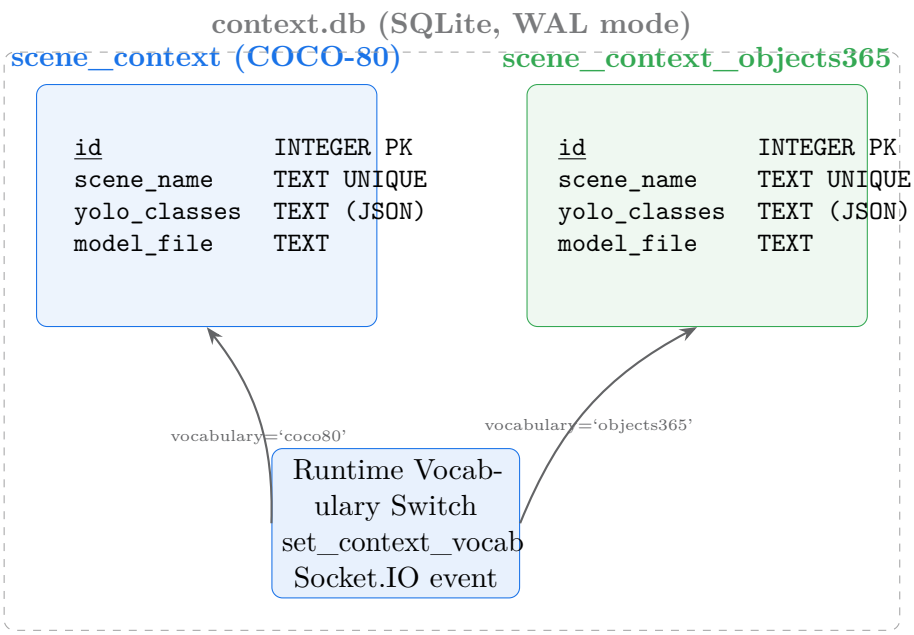

Figure 3: Dual-vocabulary database architecture. Both tables share an identical schema; vocabulary selection is controlled at runtime via Socket.IO.

Mermaid Source (context_manager.mmd)

```
erDiagram
    SCENE_CONTEXT { text scene_name UK; text yolo_classes; text
       model_file }
    SCENE_CONTEXT_OBJECTS365 { text scene_name UK; text
       yolo_classes; text model_file }
    INFERENCE_HISTORY { text switch_id UK; text scene; text mode;
       real switch_ms }
    PHASE1_HISTORY { text infer_id UK; text scene; real confidence;
        real inference_ms }
    SCENE_CONTEXT ||--o{ INFERENCE_HISTORY : "scene lookup"
    PHASE1_HISTORY ||--o| INFERENCE_HISTORY : "triggers"
```

5.3 Vocabulary Coverage Analysis

Table 4: Class coverage comparison: COCO-80 vs. Objects365 per scene.

Scene	COCO-80	Objects365	Examples unique to Objects365
living_room	14	32	carpet, lamp, pillow, blanket, curtain, magazine
kitchen	13	22	faucet, kettle, blender, gas stove, cutting board
office	10	22	monitor, tablet, printer, whiteboard, ruler, stapler
hospital	6	16	wheelchair, stethoscope, syringe, thermometer, bandage
street	12	19	SUV, street lights, traffic cone, mailbox
concert	0	11	guitar, violin, piano, drum, saxophone, microphone
Average	**9.2**	**17.5**	**≈1.9× more classes/scene**

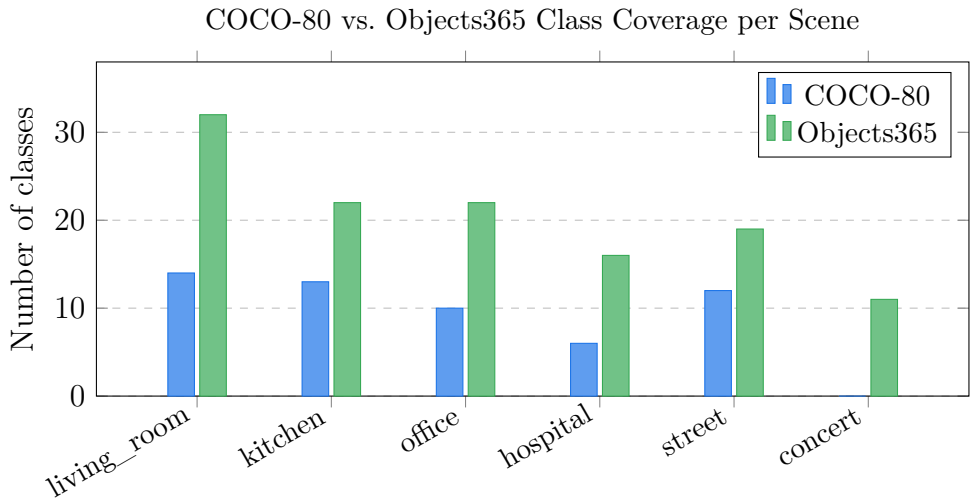

Figure 4: Objects365 provides on average 1.9× more detectable classes per scene, with especially large gains for medical, musical, and indoor scenes.

6 YOLO Model Benchmark

6.1 Model Formats Evaluated

All models are derived from **YOLOv8n (nano)** and evaluated on the Raspberry Pi 5 CPU (ARM Cortex-A76) with input size 640 × 640, test image: `bus.jpg` (4 persons, 1 bus).

Table 5: Inference benchmark results on Raspberry Pi 5 CPU.

Model File	Format	Inference (ms)	Detections	Notes
yolo26n_float16.tflite	TFLite float16	**302.0**	4p + 1b	XNNPACK delega
yolo26n_float32.tflite	TFLite float32	309.1	4p + 1b	XNNPACK delega
yolo26n.onnx	ONNX Runtime 1.24.1	420.3	4p + 1b	CPUExecutionProvi
yolo26n.pt	PyTorch (ultralytics)	460.8	4p + 1b	torch 2.10.0+cpu

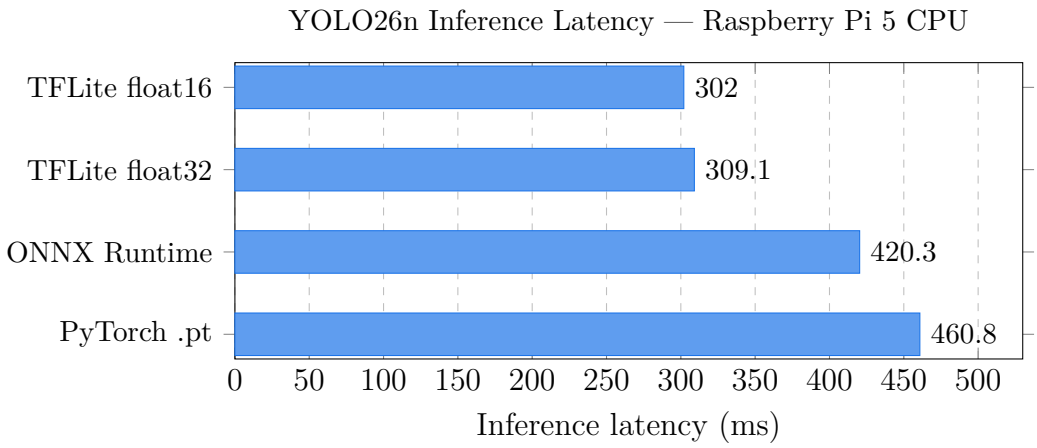

Figure 5: TFLite float16 achieves the fastest inference at 302 ms, a 35% improvement over PyTorch. XNNPACK delegate enables ARM-optimised SIMD execution for TFLite on Cortex-A76.

Key Finding

TFLite float16 is the recommended deployment format for Raspberry Pi 5. It achieves the lowest latency (302 ms) while maintaining identical detection output (4 persons + 1 bus). The ONNX Runtime is a viable alternative at 420 ms without requiring TensorFlow Lite. The full Objects365 model benchmark (365+ model variants) is available on **Google Drive** and **Google Colab** — see Section 11.1.

7 Communication Architecture

7.1 Socket.IO Event Map

Table 6: Key Socket.IO events (C = client→server; S = server→client).

Event	Dir.	Payload / Purpose
`authenticate`	C	`{password}` — unlock control commands
`move`	C	`{direction: forward|backward|left|right|...}`
`stop`	C	Emergency stop
`servo`	C	`{id: 1|2, angle: 0-180}`
`speed`	C	`{speed: 0-255}`
`led`	C	`{action, color: 0-6}`
`detection_toggle`	C	`{enabled: bool}`
`set_scene`	C	`{scene: 'kitchen'}` — manual Phase 2 override
`set_context_vocab`	C	`{vocab: 'coco80'|'objects365'}`
`alert_rules_sync`	C	Full-replace alert rules array
`status`	S	Robot state broadcast @ 4 Hz
`sensors`	S	Ultrasonic, IR, line-tracking @ 3 Hz
`detection_results`	S	YOLO results per inference frame
`phase1_result`	S	Top-5 Places365 scene predictions
`phase2_timing`	S	`{switch_id, scene, mode, db_ms, switch_ms}`
`alert_triggered`	S	Alert rule fired

7.2 Camera Streaming: MJPEG vs. WebRTC

Table 7: Camera transport mechanism comparison.

Property	MJPEG (current)	WebRTC (experimental)
Transport	TCP (HTTP chunked)	UDP (DTLS/SRTP RTP)
Codec	JPEG per frame	H.264 / VP8 / VP9
Compression	Spatial only (DCT)	Spatial + temporal
Typical bitrate	2–5 Mbps	200–600 kbps
Glass-to-glass latency	500 ms – 2 s	<200 ms
Head-of-line blocking	Yes (TCP)	No (UDP discard)
Browser decode	Software	GPU-accelerated
Implementation complexity	Low	High (aiortc, asyncio)
Status	**Production**	Experimental

The end-to-end latency models for the two transports are:

$$L_{\text{MJPEG}} = t_{\text{cap}} + t_{\text{JPEG}} + t_{\text{TCP buffer}} + t_{\text{render}} \tag{1}$$

$$L_{\text{WebRTC}} = t_{\text{cap}} + t_{\text{encode}} + t_{\text{UDP}} + t_{\text{decode}} \tag{2}$$

8 Frontend Architecture

8.1 Component Overview

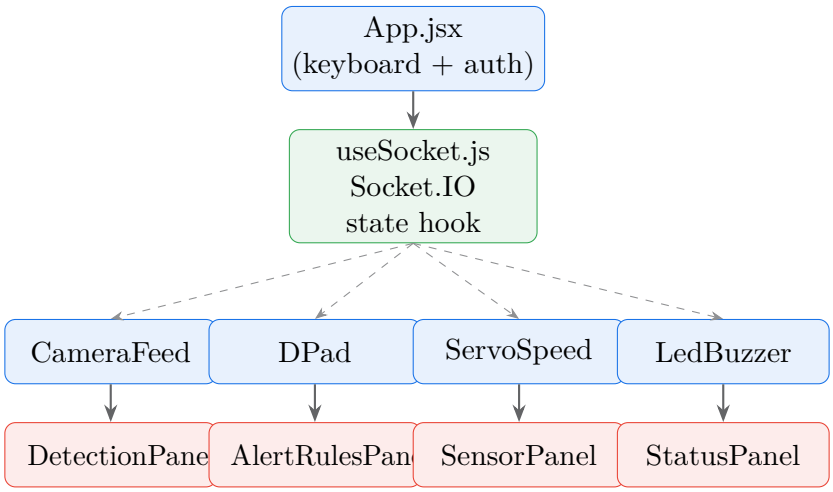

Figure 6: Frontend React component tree. `useSocket.js` distributes Socket.IO state; `App.jsx` coordinates keyboard/auth logic.

Mermaid Source (frontend_dataflow.mmd)

```
sequenceDiagram
    participant User
    participant App as App.jsx (React)
    participant Hook as useSocket.js
    participant IO as Socket.IO Server
    participant HW as Hardware (Raspbot_Lib)

    User->>App: Press WASD / touch D-pad
    App->>Hook: emit('move', {direction})
    Hook->>IO: WebSocket frame
    IO->>HW: Car.set_motor() via I2C
    IO-->>Hook: broadcast 'status' @ 4 Hz
    Hook-->>App: setStatus(newState)
    App-->>User: UI reflects motor state
```

9 Alert Rules Engine

The alert system allows the robot to autonomously respond when specific objects are detected above a configurable threshold.

Table 8: Alert rule structure.

Field	Type	Description
`id`	UUID string	Unique rule identifier
`class_name`	string	YOLO class to watch (e.g., `person`)
`count_threshold`	int	Trigger when count $\geq$ threshold
`action_type`	string	`led_color`, `led_rgb`, `buzzer_on`, `buzzer_pattern`
`action_params`	dict	e.g., `{on_ms: 200, off_ms: 200, repeats: 3}`
`enabled`	bool	Rule active flag

Rising-edge detection triggers the action after N consecutive frames meeting the threshold; falling-edge clears it after N frames below threshold. Anti-flicker via `alert_cooldown` counter prevents rapid on/off oscillation.

10 Docker Deployment Architecture

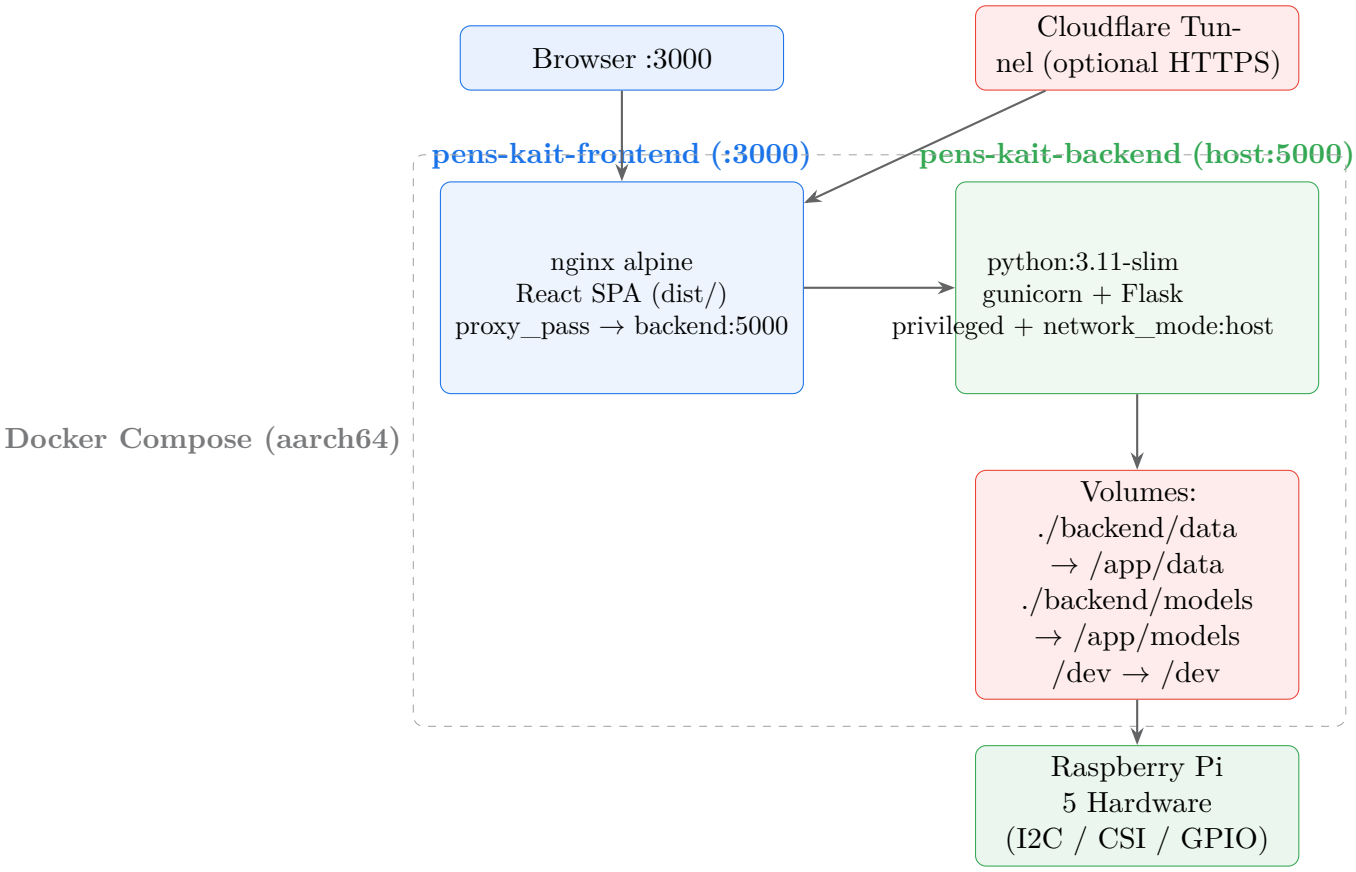

Figure 7: Docker Compose deployment. Backend uses `network_mode: host` and `privileged: true` for hardware access. Volumes persist SQLite databases and model files across container restarts.

Mermaid Source (docker_deployment.mmd)

```
graph TB
  subgraph DC["Docker Compose (aarch64)"]
    FE["pens-kait-frontend :3000\nnginx + React SPA\nproxy_pass ->
        backend:5000"]
    BE["pens-kait-backend :5000\ngunicorn + Flask\nprivileged: true
        , network_mode: host"]
  end
  FE <-->|proxy_pass| BE
  BE <-->|"smbus2 I2C / rpicam-vid"| HW["Raspberry Pi 5 Hardware"]
  BROWSER -->|"HTTP :3000"| FE
  CF["Cloudflare Tunnel (optional)"] --> FE
```

10.1 Build & Run Commands

Docker Compose Commands

```
# Build and start both containers
sudo docker compose up --build

# Run detached (production)
sudo docker compose up -d

# Access dashboard: http://<pi-ip>:3000

# Backend local dev
cd backend && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && cp .env.example .env
python app.py   # http://localhost:5000

# Frontend local dev
cd frontend && npm install && npm run dev  # http://localhost:5173
```

11 Storage Limitation & Procurement Plan

Current Storage Constraint

The system currently runs on a **microSD card** with limited available capacity. The following storage demands exceed what is currently available on-device:

- **Objects365 frozen model set**: 365 scene-specific YOLO model variants, each exported in multiple formats (PyTorch, ONNX, TFLite float32, TFLite float16). Estimated total: **15–50 GB**.
- **YOLO training outputs** (`runs/`): Checkpoints, tensorboard logs, and prediction outputs from model fine-tuning.
- **NCNN and OpenVINO format caches**: Compiled models for additional inference backends.

11.1 Interim Solution: Google Drive & Google Colab

While hardware procurement is pending, all model training, format conversion, and benchmarking for the full Objects365 model set is performed in **Google Colab** (A100/T4 GPU environment) and stored on **Google Drive**. This includes:

- Full benchmark results table (all 365 scene models × 4 formats)
- Training logs and validation curves
- Exported model files (.pt, .onnx, .tflite float32/float16)
- Performance comparison notebooks

Access to Results

Google Drive: Full model set, benchmark CSVs, and exported models
Google Colab: Benchmark notebooks, training scripts, format conversion pipeline

Links will be shared in the seminar session.

11.2 Procurement Plan

Table 9: Recommended storage upgrade options for Raspberry Pi 5.

Option	Interface	Capacity	Notes
M.2 NVMe (M.2 HAT+)	PCIe 2.0 ×1	256 GB – 2 TB	Best performance, ∼400 MB/s
USB 3.0 SSD (external)	USB 3.0	480 GB – 2 TB	Portable, no HAT required
High-speed microSD (A2)	UHS-I / SDR104	128–512 GB	Minimal modification

The **M.2 NVMe via M.2 HAT+** is strongly recommended: it mounts directly to the Raspberry Pi 5 PCIe connector, offers sequential read speeds of ∼400 MB/s (vs. ∼90 MB/s on microSD), and provides ample capacity for all model variants and training runs.

12 Performance Summary

Table 10: Key performance metrics vs. targets.

Metric	Target	Measured	Status
Control latency (keypress → motor)	<100 ms	∼50–80 ms (LAN)	✓
Camera capture FPS	30 fps	24–30 fps	✓
YOLO inference (yolo26n, TFLite f16)	fastest achievable	302 ms	✓
YOLO inference (yolo26n, ONNX)	—	420 ms	—
YOLO inference (yolo26n, PyTorch)	—	461 ms	—
Phase 1 GoogLeNet inference	∼1 Hz rate	50–100 ms	✓
Phase 2 switch (classes mode)	<3 s	5–20 ms	✓
Phase 2 switch (model mode)	<3 s	200–500 ms	✓
Backend memory footprint	<1.5 GB	∼0.8–1.2 GB	✓

13 Current Limitations & Future Work

L1. CPU-only inference: YOLO runs on the ARM Cortex-A76 without hardware acceleration. Exploratory Vulkan compute work (`backend/vulkan_test.py`) targeting the VideoCore VII GPU is in progress. A successful implementation could reduce inference latency from 302 ms to <50 ms.

L2. Storage constraint: As described in Section 11, NVMe SSD procurement is required to deploy the full Objects365 model set on-device.

L3. WebRTC in production: The aiortc integration is experimental; ICE candidate handling and TURN server configuration need finalisation for non-LAN deployments.

L4. NCNN format: NCNN-format models (`yolo26n_ncnn_model/`) are generated but not yet integrated into the inference pipeline — a promising path for further latency reduction.

L5. YOLOWorld fine-tuning: The current Objects365 vocabulary mapping uses pre-trained class embeddings. Fine-tuning YOLOWorld on domain-specific data (e.g., medical or industrial scenes) is planned.

14 Conclusion

PENS-KAIT 2026 demonstrates a complete, deployable **context-aware robot control platform** on Raspberry Pi 5. The core contributions presented in this seminar report are:

- A working **two-phase detection pipeline** combining Places365 scene recognition with dynamic YOLOWorld class switching.
- A **dual-vocabulary context database** (COCO-80 + Objects365) providing 1.9× greater class coverage per scene.
- A **benchmark** showing TFLite float16 achieves 302 ms inference — 35% faster than PyTorch — on Raspberry Pi 5 CPU.
- A **production-ready Docker deployment** with authenticated Socket.IO control, MJPEG streaming, alert rules, and a full React dashboard.
- An **experimental WebRTC subsystem** reducing streaming latency from 500 ms–2 s (MJPEG) to <200 ms.

The immediate next steps are NVMe SSD procurement to enable full on-device model deployment and Vulkan GPU compute integration for sub-50 ms YOLO inference.

A File Structure Reference

```
PENS-KAIT 2026/
+-- backend/
|   +-- app.py                  # Main server (Flask + Socket.IO, 1975 lines)
|   +-- context_manager.py      # Scene-to-class DB manager (621 lines)
|   +-- requirements.txt
```

```
|   +-- Dockerfile
|   +-- data/
|   |   +-- context.db          # SQLite: scene_context + scene_context_objects365
|   |   +-- access.db           # SQLite: access_log + inference_history + phase1_history
|   |   \-- places365.csv       # Places365 scene category CSV (seed data)
|   \-- models/
|       +-- phase1_model.py     # GoogLeNet Places365 inference wrapper
|       +-- yolo26n.pt/.onnx    # YOLOv8 nano (PyTorch / ONNX)
|       +-- yolo26n_saved_model/ # TFLite float32 + float16
|       +-- yolov8s-worldv2.pt  # YOLOWorld (dynamic class setting)
|       \-- Phase1/             # GoogLeNet Caffe weights + prototxt
+-- frontend/
|   +-- src/
|   |   +-- App.jsx             # Main app (keyboard, auth)
|   |   +-- hooks/useSocket.js  # Socket.IO state management
|   |   \-- components/         # CameraFeed, DPad, ServoSpeed, DetectionPanel, ...
|   \-- Dockerfile              # Multi-stage: node:20 -> nginx:alpine
+-- docker-compose.yaml
+-- docs/
|   +-- seminar/                # This document
|   |   +-- main.tex
|   |   +-- build.sh
|   |   \-- diagrams/*.mmd      # Mermaid source files
|   \-- system_progress_report_*.md/.pdf
\-- PRD.md                      # Product Requirements Document
```

B Environment Variables

Table 11: `backend/.env` configuration variables.

Variable	Default	Description
`ADMIN_PASSWORD`	(required)	Dashboard unlock password
`CAMERA_TYPE`	`AUTO`	`CSI` | `USB` | `AUTO`
`TURNSTILE_SECRET_KEY`	(optional)	Cloudflare Turnstile secret
`TURNSTILE_SITE_KEY`	(optional)	Cloudflare Turnstile public key